ChatGPT

Executive Summary

We will build a single full-stack Next.js (TypeScript) application that serves a **public corporate website** and a **protected staff dashboard**. The landing pages and quotation form will use modern UI (Tailwind CSS + shadcn/ui) while server-side routes handle data. A Supabase PostgreSQL backend (with Auth and RLS) will store customers, quotation requests, invoices and payments. Accepted quotations auto-generate GST-compliant invoices (calculating CGST/SGST for intrastate or IGST for interstate supplies) and handle payments. We will develop this in phases: Phase 1 sets up the project foundation, public site and request form (no persistence yet); later phases add Supabase schemas, auth, quotation/invoice workflows, PDF/email integration and deployment.

Architecture Diagram and Workflow

Rendered Mermaid diagram 1

- **Public Users:** Access the **Next.js app** for the landing pages and **"Request a Quotation" form**. This form submission calls a server action (or API route) to store the request. No user login is required for public visitors.
- **Staff (Admin):** Login via Supabase Auth. Authenticated users see an **internal dashboard** in Next.js (protected by server-side checks) to view/manage quotation requests, create quotations, generate invoices, and record payments. All business logic (tax calculations, PDF/invoice generation, email sending) runs on the server side (using Next.js Route Handlers or Server Actions【46†L82-L86】).
- **Data Flow:** The Next.js server connects to Supabase's Postgres database and Auth service. When a public form is submitted, a row is inserted into the "requests" table. Staff actions (prepare quote, issue invoice, log payment) update the DB. PDFs and emails are generated on the server: for example, a "Send Quotation" action creates a PDF (via an HTML-to-PDF library) and triggers a transactional email (using a service like Resend).
- **Tools:** We'll use **Next.js** as the full-stack framework (for React, SSR, server actions, and API routes)【22†L61-L64】【46†L82-L86】. **Tailwind CSS** provides a utility-first styling framework【20†L12-L13】, and **shadcn/ui** offers reusable accessible components (buttons, forms, tables) that integrate with Tailwind【29†L15-L18】. Supabase supplies the **Postgres database** (for queries, joins, indexing) and **Authentication** (user accounts, JWTs) in one platform【53†L8-L10】【53†L13-L16】. This stack (Next.js + Supabase) allows us to build a secure full-stack app without managing our own servers. Finally, we will host on **Vercel** (seamless with Next.js), and integrate email/PDF tools as needed.

Workflow Mermaid: (Quotation lifecycle)

```
flowchart TD
  subgraph PublicFlow[Public (no auth)]
    Customer(Customer) -- fills inquiry --> Site{Website Form}
    Site -- insert request --> DB[(Quotation_Requests)]
```

```
  Site -- ack email --> Customer
end
subgraph AdminFlow[Staff (Supabase Auth)]
  Staff -- login --> SupabaseAuth
  Staff -- view/edit --> Dashboard(Admin UI)
  Dashboard -- create quotation --> DB[(Quotations)]
  Dashboard -- send quotation (email/PDF) --> Customer
  Customer -- (accept) --> Dashboard
  Dashboard -- create invoice --> DB[(Invoices)]
  Dashboard -- record payment --> DB[(Payments)]
end
```

This shows that a customer submits a request (stored in Supabase), staff create/send a quotation (tracked in the DB), and upon acceptance staff generate an invoice and record payments. All private steps (in `AdminFlow`) require the user to be authenticated.

Technology Stack and Rationale

- **Next.js (App Router + TypeScript):** A robust React framework for production. Next.js enables server-side rendering, API routes (Route Handlers) and Server Actions, which let us "run server code by calling a function" without a separate backend【46†L82-L86】. It has built-in performance optimizations and works seamlessly with Vercel. We will use the new App Router (Next.js 13/14) for nested layouts, dynamic routes, and React Server Components as needed.
- **TypeScript:** Provides static typing for reliability, reducing runtime errors in a business app (calculations, data flows). We'll enable strict mode to catch bugs early.
- **Tailwind CSS:** A **utility-first** CSS framework that allows rapid, consistent styling directly in markup【20†L12-L13】. It's highly customizable and works well with React. We avoid long CSS files by using Tailwind classes (e.g. spacing, typography, colors). Tailwind's design system will use the company's navy blue (#10243B), gold (#F4B323) and neutrals.
- **shadcn/ui:** A community component library of **composable, accessible components** built on Tailwind【29†L15-L18】. It includes common UI elements (buttons, forms, dialogs, tables) with sensible defaults, accelerating development and ensuring consistency. We'll use it to assemble the landing page sections and form components.
- **Lucide Icons:** (or Heroicons) for vector icons. We'll incorporate them via React components or SVGs for visual cues (e.g. process steps, contact icons).
- **Supabase:** Our backend platform (Postgres database + Auth + storage)【53†L8-L10】【53†L13-L16】. It provides:
- *PostgreSQL Database:* A relational DB hosted by Supabase, with standard SQL for our quotation/invoice data. We'll define tables (quotations, invoices, etc.) and use SQL indexes for performance.
- *Auth and RLS:* Built-in user authentication (email/password or SSO) and row-level security. We'll define user roles (staff vs anon) in Supabase and write RLS policies to ensure only authorized users can query sensitive tables【53†L13-L16】【40†L278-L284】. For example, the `anon` role will only be allowed to INSERT new quotation requests (and not read any tables), while authenticated staff can CRUD on quotations and invoices.

- *Realtime and Storage:* Although not immediately needed, Supabase can host images or documents. (We might store generated PDFs or uploaded files in Supabase Storage with secure access if needed.)
- **Transactional Email:** We'll use a service like **Resend** or similar to send emails (for quoting and invoicing). Resend provides a Node.js SDK to send HTML emails. This avoids managing SMTP ourselves. Emails (e.g. "Your quotation is ready") will be triggered by server-side actions.
- **PDF Generation:** On the server, we will render HTML/CSS invoices into PDF. A common approach is using a headless browser (Puppeteer) or a library like `html-pdf` or `pdfkit`. For example, the invoice component could be rendered with React then sent through Puppeteer to produce a printable PDF.
- **CI/CD / Deployment:** We'll connect the GitHub repo to **Vercel** for automatic deployments on commits. Environment variables (Supabase URL, keys, Resend API key, etc.) will be set in Vercel's dashboard and never checked into source control. Vercel handles HTTPS and scaling. We'll also set up preview deployments for each PR.

This stack (Next.js+Tailwind on front, Supabase on back) is widely adopted for modern web apps【22†L61-L64】【53†L8-L10】. Next.js is "trusted by some of the biggest names on the web" and gives us production-grade performance【22†L61-L64】, while Supabase gives us a **full Postgres database** plus Auth and RLS out of the box【53†L8-L10】【53†L13-L16】.

Database Schema (Supabase)

We propose the following core tables (others like user profiles or audit logs are included where noted):

Table	Key Columns & Fields	Indexes/RLS Notes
customers	id (PK, uuid), name, email, phone, address, `gstin` (optional), created_at	Unique index on email. RLS: authenticated users only. Each row = a customer/organization from a request.
requests	id (PK), customer_name, business_name, contact_name, contact_email, contact_phone, customer_gstin (opt), category, details, state, created_at	RLS: Allow `anon` to INSERT only (public form), authenticated to SELECT/UPDATE. No table read by anon.
quotations	id, request_id (FK), customer_id (FK), status (e.g. Draft, Sent, Accepted, Rejected), total, valid_till, terms, created_by, created_at	Index on status/date. RLS: only staff (authenticated) can SELECT/ INSERT/UPDATE.
quotation_items	id, quotation_id (FK), description, `hsn_sac` (code), quantity, unit_price, discount_pct, cgst_rate, sgst_rate, igst_rate, taxable_value, total	RLS inherited via join with quotations. No grants to anon.

Table	Key Columns & Fields	Indexes/RLS Notes
invoices	id, quotation_id (FK), invoice_number (unique), issue_date, due_date, cgst_amt, sgst_amt, igst_amt, total, status (Paid/Unpaid/Partial)	Unique index on invoice_number. RLS: staff-only.
invoice_items	id, invoice_id (FK), description, `hsn_sac`, quantity, unit_price, cgst_rate, sgst_rate, igst_rate, taxable_value, total	RLS via invoices.
payments	id, invoice_id (FK), amount, paid_at, method, reference	Index on invoice_id. RLS: staff-only.
audit_logs	id, user_id, table_name, record_id, action (INSERT/UPDATE/DELETE), changed_data (JSON), timestamp	RLS: staff-only; the service role (invoked via triggers) can insert audit records.

- **customers vs requests:** We separate a *customer* record from each *request*. If a known customer submits multiple requests, they share an ID. Customers may have GSTIN and address used for invoicing.
- **Quotation vs Invoice:** A quotation becomes an invoice once accepted. We store them in separate tables, linked by `quotation_id`. Invoice numbers (sequential) are unique. Taxes (cgst_amt etc.) are pre-calculated and stored.
- **RLS policies:** We will **enable RLS on all tables** and revoke default grants【40†L278-L284】. For example: only allow `anon` role to INSERT into `requests` with a policy (`TO anon FOR INSERT`), and allow `authenticated` to read requests. For `quotations/invoices`, policies will ensure only staff (authenticated users) can read or write. The Supabase docs emphasize that new tables grant **all privileges by default** and must be restricted manually【40†L278-L284】. We will grant only the needed privileges per role.
- **Indexes:** We will add indexes on foreign keys and commonly filtered fields (status, dates). For instance, index `quotations(status)` and `invoices(invoice_number)` for quick lookup.
- **Data integrity:** Foreign keys enforce that items reference existing quotations/invoices. Totals and tax amounts should be computed server-side or via database functions (never trusting client input).

API Routes and Server Actions

We'll use Next.js App Router's conventions for pages and API endpoints. Key routes and actions include:

- **Public Routes (no auth):**
- `GET /` (Home page) – static content, no auth.
- `GET /about`, `/services`, `/machinery`, `/contact` – static/info pages.
- `GET /request-quotation` – renders the quotation request form.
- `POST /request-quotation` – server action or API route that validates the form, inserts a new row into `requests`. *Auth:* none (anon). It should perform spam checks (e.g. CAPTCHA).
- **Staff Routes (auth required):**
- `GET /admin` (dashboard root) – redirects to login if not signed in (via Supabase Auth).
- `GET /admin/requests` – list pending requests. (Requires `authenticated` session).

- `GET /admin/requests/[id]` – detail view of a request with buttons to create a quotation.
- `POST /admin/quotations` – create a new quotation (with line items) from a request. *Auth:* staff.
- `GET /admin/quotations/[id]` – view/edit a quotation.
- `POST /admin/quotations/[id]/send` – generates PDF & emails the quotation to customer. *Auth:* staff.
- `POST /admin/quotations/[id]/status` – update status (e.g. mark accepted). *Auth:* staff.
- `POST /admin/invoices` – generate an invoice from an accepted quotation. *Auth:* staff.
- `GET /admin/invoices/[id]` – view invoice.
- `POST /admin/invoices/[id]/payments` – record a payment. *Auth:* staff.
- **Auth & Misc:**
- `/api/auth/*` – Supabase-managed routes for sign-in/sign-out callbacks.
- **Server Actions:** We'll use Next.js Server Actions for form submissions and side-effects where possible. For example, the quotation request form can use a Server Action that inserts into Supabase. For emailing/PDF, a Server Action or API Handler can orchestrate the external calls.

Each protected route will verify the user's JWT with Supabase; we can use Supabase's server-side client (with service key in environment) or the new Next.js Edge Middleware for auth. All API requests from the browser to our server should use secure, server-checked calls (never embed service keys in client code).

Indian GST Calculation Rules

Our system must calculate GST correctly on each invoice. Key rules to implement:

- **Place of Supply:** If **seller and buyer are in the same state**, charge **CGST + SGST** (state and central share). If **in different states**, charge **IGST**. We determine this by comparing the company's registered state (from its GSTIN) to the customer's shipping state. For domestic B2B/B2C supplies, this is standard. (Exports or SEZ sales would be IGST@0%, but we assume only domestic transactions for now.)
- **GSTIN validation:** Indian GSTIN is 15 characters (2-digit state code + 10-char PAN + 1 entity code + 1 check digit). On the public form, if a GSTIN is provided, we'll validate its format (e.g. via regex). However, we will not require it or check it against the GST portal. It's optional.
- **HSN/SAC Codes:** Each line item (machinery or service) should include an HSN (for goods) or SAC (for services) code. We will allow staff to enter it. The database fields `hsn_sac` store these. The tax rates can vary by HSN (some goods may have 5%, 18%, etc). We should allow specifying the GST rate per item. At invoice creation, CGST/SGST/IGST amounts are computed as:
- `taxable_value = unit_price * qty - discount`
- If intrastate: `cgst_amount = taxable_value * cgst_rate`, `sgst_amount = taxable_value * sgst_rate`. (Typically cgst_rate = sgst_rate = half of the total tax rate).
- If interstate: `igst_amount = taxable_value * igst_rate`. (Typically igst_rate = total tax rate).
- **Discounts:** If any line item has a discount (fixed or percentage), apply it before tax. We should compute `taxable_value` post-discount and then compute each tax component.
- **Rounding:** Indian GST rules allow invoice totals to be rounded to the nearest rupee. We'll sum line-item totals (which may have paise) and then round the final CGST, SGST, IGST and invoice total to two decimal places (or ideally integer rupees). We can implement rounding in the server code (e.g. `Math.round(value * 100) / 100`). We must ensure the sum of parts equals the grand total up to rounding.

- **Edge Cases:** Ensure zero-rated or exempt items are handled (if machinery has 0% GST, set all rates to 0). If a customer has no GSTIN, still issue a bill with B2C rates (same logic). We should also consider if an invoice crosses state borders and still is intrastate (very rare for physical goods).

Unfortunately, official GST calculation rules are complex, but for our purposes: **intrastate = CGST+SGST, interstate = IGST**. We will build these rules into our invoice-generation code. (No user input should override these calculations.)

PDF Generation Approach

We need to generate professional PDF documents for quotations and invoices. A typical approach: design React components or HTML templates matching the company's invoice style, then render them to PDF on the server. For example:

- Use a Node library (like **Puppeteer** or **Playwright**) to launch a headless Chrome. We create an invoice page in Next.js (server render) and instruct Puppeteer to navigate to that page (passing query params or session) and `page.pdf()` to capture it.
- Alternatively, use a library like `pdfkit` or `jsPDF` programmatically building the document, but HTML-to-PDF (Puppeteer) ensures precise styling with CSS/Tailwind.
- This PDF generation will run in a Next.js Route Handler (server-side). We'll then either save the PDF to Supabase Storage or attach it to an email (using the email service's attachment feature). If storing, we can use Supabase Storage buckets (access restricted via signed URLs).
- We should ensure any dynamic data (like totals) is fully calculated before PDF generation. The PDF should have a sequential invoice number (auto-generated and stored in DB) and the company's header (logo, address, GSTIN).

No need for a front-end PDF; this is purely a back-end process. We should be careful to not time out long renders (keep the page simple).

Email/Notification Flow

Transactional emails will notify relevant parties:

- **New Request Alert:** After a public user submits the quotation form, optionally send an email to the sales team (staff) saying "New quotation request received" with summary. This ensures staff know to log in. We may use a service account and a pre-set company email.
- **Send Quotation:** When staff finalizes a quotation and clicks "Send to client", the server generates the quotation PDF and emails it to the customer's email with a message. The email content and PDF are crafted from templates. (This can be done in one action: generate PDF, call `Resend.sendEmail` with HTML and PDF buffer.)
- **Invoice Issued:** Similarly, when an invoice is created, we email it to the client (and optionally a copy to accounts).
- **Payment Received:** If a payment is recorded, optionally send a receipt or payment confirmation to the client. This might be email or could just log internally.

For email service, Resend (https://resend.com) or similar SMTP/SendGrid can be configured. We'll keep templates in code or storage. All email sending happens in server code using secure API keys (no emails are sent from the client).

Security Considerations

- **RLS and Auth:** As noted, every table with sensitive data will have Row-Level Security enabled 【40†L278-L284】. We will revoke default grants and create explicit policies. Only the `service_role` key (kept on server) can bypass RLS. On client side, use only the Supabase anon/public key or session, which is limited by RLS.
- **Secrets:** Never expose service or API keys in client code. Store them in environment variables. For example, the Supabase service_role key and email service API key only in server env (and Vercel secrets).
- **Rate Limiting & Spam:** The public form is a vector for spam. We should implement a CAPTCHA (Google reCAPTCHA or hCaptcha) on the form page and verify it server-side before inserting the request. Additionally, we can use a simple rate-limit (e.g. no more than X submissions per IP per hour) using middleware or a third-party rate-limit library. The same applies to admin APIs (logins should be rate-limited by Supabase or Firebase Auth automatically).
- **Input Validation:** Use **Zod** or similar on all form inputs, especially email/phone/GSTIN formats. This prevents injection and ensures data quality. On the server, always use parameterized queries or Supabase client calls (to avoid SQL injection).
- **HTTPS and CSP:** Deploy on Vercel which provides HTTPS. We'll also set Content Security Policy headers to allow scripts/styles only from our domains (Next.js can do this via `next.config.js`).
- **Audit Logs:** The `audit_logs` table should record every create/update/delete on quotes/invoices (triggered by backend code). This helps track user actions for compliance. We'll ensure the service user (with permission) writes these logs; RLS can even prevent regular users from modifying audit entries.

Deployment & CI/CD Checklist

Before going live, we will ensure:
1. **Version Control:** The project is in GitHub. Include `.gitignore` for node_modules, `.env.local`, etc.
2. **Environment Setup:** In Vercel's dashboard, set environment variables:

- `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY` (for client).
- `SUPABASE_SERVICE_ROLE_KEY` (for server-side use only).
- `RESEND_API_KEY`, `EMAIL_FROM_ADDRESS`, etc.
- **Database Configuration:** Create a Supabase project. Apply migrations or SQL for tables above. Enable RLS on each table and create policies. Verify grants (revoke anon grants).
- **Authentication Setup:** Configure Supabase Auth (enable email sign-in, possibly set allowed domains if needed). Create at least one staff user.
- **Testing:** Write unit tests for tax calculations and server actions. In Antigravity, we can set up `supabase test` suites for RLS.
- **Build & Lint:** Ensure `npm run lint` and `npm run build` pass without errors (TypeScript type-check, ESLint, Tailwind JIT scan).

- **Backup:** Enable Supabase automated backups or plan manual dumps (Supabase has automatic daily backups by default). Ensure we can restore if needed.
- **Monitoring:** Set up error logging (e.g. Sentry) on production, or Supabase logs.
- **Domain & TLS:** Point the custom domain (if provided) to Vercel, ensuring TLS.
- **Legal Pages:** Placeholder routes `/privacy-policy` and `/terms-of-service` should exist (content to be added by company lawyers).

No credentials (like the Supabase URL/keys) should be in code; always use `process.env`. Use Vercel's Environment Variables for deploys.

Phase 1 Implementation Prompt for Antigravity

The first Antigravity prompt will focus only on **Project Foundation and Public Site (Phase 1)**. We will **not** build any quotation/invoice logic yet. The prompt should:

- Initialize a Next.js App Router project with TypeScript and Tailwind CSS.
- Set up the basic folder structure (pages/app directory, components, styles).
- Import the provided `Santhaya Associates` logo (assume file is in workspace, use `<Image>` or `<img>` as appropriate).
- Create the homepage with the hero, about, services, workflow, contact sections as described. Do **not** hardcode any fake data (use placeholders or props).
- Create a responsive navigation (logo + menu + request button).
- Implement `/request-quotation` page with a form and front-end validation (using Zod or simple HTML constraints). Do *not* connect it to a database yet — we'll mark the submission as "would send to backend later".
- Ensure mobile responsiveness. Use Tailwind classes for layout. Use shadcn/ui components for form and UI blocks.
- Include meta tags, alt text, and ensure accessibility (aria-labels).
- Lay out any legal page links (even if placeholder content).
- The prompt should instruct Antigravity to **inspect the workspace** first (in case any files exist) and **explain structure before coding**.
- Remind it to **save changes and run lint/build** after implementation.

Antigravity Prompt Text (copy into AG):

```
# Project: Santhaya Associates Pvt. Ltd. – Phase 1

Act as a senior full-stack React/Next.js engineer and UI/UX designer. We are
starting a project for Santhaya Associates (an Indian construction & machinery
company). Build the **project foundation and public-facing site** only
(Phase 1). Do **not** implement quotation or billing logic yet.

## 1. Inspect the workspace
- If the project already has files or configuration, review them. If empty,
initialize a new Next.js project.
- Use `npx create-next-app@latest` with **App Router**, **TypeScript**, and
**Tailwind CSS**. Ensure we have eslint and Prettier or the default formatting.
```

2. File structure
Explain the planned structure (e.g., `app/`, `app/page.tsx`, `components/`, `styles/`, `public/`). Mention where the logo file (provided) should go (e.g., `public/logo.png`).

3. Tech setup
- Use Tailwind CSS and include shadcn/ui.
- Configure `tailwind.config.ts` with the company colors: navy `#10243B`, gold `#F4B323`, white, and neutrals.
- Use React components, Tailwind classes, and shadcn/ui components for UI elements.

4. Brand and styling
- Place the provided Santhaya Associates logo prominently (in header). Do not recreate or distort it.
- Use a clean, modern, corporate design. Navy blue and gold should guide headings/buttons. Ensure good contrast (WCAG AA+).
- Use a professional serif or sans-serif font (e.g. Inter or Roboto). Shadcn/ui defaults are fine.

5. Public landing page (`app/page.tsx`)
Create a responsive homepage with the following sections:

- **Header/Nav:** Logo on left, links to Home, About, Services, Machinery Supply, Contact. A “Request a Quotation” button on right. On mobile, collapse into a hamburger menu.
- **Hero:** Full-width hero section with a background image (industry machinery or construction site, if available). Overlay with a bold headline: “Building Today. Powering Tomorrow.” Subtext: “Reliable tender participation and machinery supply solutions for construction and manufacturing.” Two buttons: “Request a Quotation” (primary) and “Explore Our Services” (secondary).
- **About Us:** Brief intro paragraph (use placeholder text) about the company’s focus (construction, machinery supply). Show a few bullet icons or an image if appropriate.
- **Services:** Three cards side-by-side (stacked on mobile):
  1. Government & Private Tenders (icon of government building)
  2. Construction Machinery Supply (icon of crane)
  3. Manufacturing Machinery Supply (icon of factory/gear)
  Each card: icon, title, a short description, and a “Inquire” button.
- **How We Work:** A 4-step infographic or horizontal list with icons/text: “1. Share requirements → 2. Discuss specs & pricing → 3. Receive a quotation → 4. Coordinate next steps.” (Use icons from Lucide or shadcn/ui).
- **Why Work With Us:** A section of key benefits (e.g. "Industry expertise, Clear quotations, Responsive support"). Use check icons or simple statements.
- **Contact/Call-to-Action:** Repeat a “Request a Quotation” button and provide placeholders for contact info: phone, email, address, GSTIN. Put them in the

footer or a dark section. Include links to Privacy Policy and Terms (create empty pages at `/privacy-policy` and `/terms` with a notice).

Ensure all text is semantic (use `<h1>`, `<p>`, `<ul>`, `<a>`, etc.). Use alt text for images and aria-labels for buttons.

6. Request Quotation page (`app/request-quotation/page.tsx`)
- Create a route `/request-quotation` with a form (use shadcn/ui Form components or HTML `<form>`).
- Form fields: Customer/Business Name, Contact Name, Email, Phone (+91 prefix), GSTIN (optional), Category (select: Construction Tenders, Construction Machinery, Manufacturing Machinery, Other), Details (textarea), Delivery State (select), Additional Notes, and a privacy checkbox.
- Use client-side validation (e.g. required fields, email format, 10-digit phone, GSTIN pattern if filled). You can use Zod for schema validation on submit. Show inline error messages.
- On submit, for Phase 1 simply prevent default and show a ****success message**** like “Thank you! Your request has been submitted.” Do NOT actually send or save (mention in code comment “// TODO: integrate with Supabase API in Phase 2”).
- Include a honeypot or basic spam measure (e.g. hidden field trap) to demonstrate spam-thought.

7. Accessibility & Quality
- Ensure focus states are visible, color contrasts meet WCAG standards, and semantic HTML is used.
- Test at mobile widths: the layout should adapt (nav collapses, sections stack).
- Set a proper `<title>Santhaya Associates — Home</title>` and meta description.
- Optimize images (e.g. use Next.js `<Image>` with appropriate `sizes`).
- Lint (`npm run lint`) and fix any warnings/errors. Type-check as well.

8. Final Report
After implementation, provide:
1. Lint/type-check/build results (no errors).
2. Screenshots or notes on homepage and form at different screen sizes.
3. List of files created/changed.
4. Next steps: (e.g. “Add Supabase and back-end integration in Phase 2, including database schemas and auth”).

Begin by confirming the workspace initialization and outlining the folder structure. Then proceed to build only the landing page and `/request-quotation` page as described.

****Important:**** Do not attempt any quotation/invoice logic yet. Focus purely on the static site UI and form UX for Phase 1.

Implementation Checklist

- **Tech Setup:** Initialized Next.js (App Router) + TypeScript + Tailwind + shadcn/ui.
- **Project Structure:** `app/` folder with `page.tsx`, `layout.tsx`, CSS config, plus `components/` and `public/` (contains logo).
- **Landing Page:** Responsive header/nav, hero, about, services, steps, benefits, contact sections built with Tailwind & shadcn components. Logo included.
- **Request Form:** `/request-quotation` page with form, validation (email/phone/GSTIN regex). Success/error UI without backend.
- **Accessibility:** Semantic HTML, alt text, ARIA labels, focus styles, color contrast.
- **Meta/SEO:** Titles, meta description, sitemap or robots (optional).
- **Lint & Build:** Ensure no errors. Commit initial code to GitHub.

Estimated file structure (example):

```
/SanthayaAssociates/
├── app/
│   ├── layout.tsx
│   ├── page.tsx            # Home page
│   ├── request-quotation/
│   │   └── page.tsx        # Quotation form
│   └── (other sections: about/, contact/ pages as needed)
├── components/
│   ├── Header.tsx
│   ├── Hero.tsx
│   ├── ServiceCard.tsx
│   └── QuotationForm.tsx
├── styles/
│   ├── globals.css
│   └── tailwind.config.js
├── public/
│   ├── santhaya-logo.png   # Provided logo image
│   └── (placeholder images)
├── tsconfig.json
├── next.config.js
└── package.json
```

The first Antigravity prompt text (above) can be pasted into the AG interface to generate the Phase 1 code. This completes Phase 1; the next phase will set up Supabase, define the database schema (quotations, invoices, etc.), and enable server-side integration.

Sources: Next.js documentation【22†L61-L64】【46†L82-L86】, Tailwind CSS docs【20†L12-L13】, Supabase platform docs【53†L8-L10】【53†L13-L16】【40†L278-L284】, and best practices for Indian GST invoicing.
